

智能旅行助手项目复盘与优化计划

一、文档目的

本文档用于对“智能旅行助手”学习与开发过程进行阶段性复盘，梳理当前项目已经完成的能力、现有架构、与官方第十三章项目的对照方向，以及后续产品化优化计划。

目前项目已经从最初的命令行小程序，逐步扩展为一个具备多工具调用、项目记忆、版本快照、报告导出和 Web 雏形的智能旅行规划 Agent 原型。后续工作的重点不再只是继续增加工具，而是进一步提升系统结构、数据可靠性、交互体验和测试稳定性。

二、项目当前定位

当前智能旅行助手可以定位为：

一个基于 LLM + Tools + Memory + Storage + Export + Web API 的智能旅行规划 Agent 原型。

从学习角度看，它已经完整体现了 Agent 的核心思想：大模型不再单独凭经验生成行程，而是结合地图、天气、预算、POI、路线、酒店、交通、开放时间、城市活动等工具结果进行综合规划。

从产品角度看，它已经从简单 Demo 进入“产品雏形”阶段。系统已经具备旅行规划产品的基本闭环：用户输入需求、生成行程、多轮修改、保存快照、恢复上一版、查看记忆、导出报告、Web 页面展示。

但从真正可上线产品角度看，它仍然存在数据源稳定性不足、地点结构化不够、缺少用户确认机制、缺少完整测试集等问题。因此，当前更适合定位为“学习型 Agent 项目 + 产品原型”，而不是成熟商业化旅行产品。

三、项目已完成能力总结

3.1 核心 Agent 流程

项目已经形成较完整的 Agent 工作流：

```
graph TD
    A[用户输入旅行需求] --> B[工具规划]
    B --> C[工具计划校验与补全]
    C --> D[工具执行]
    D --> E[大模型综合工具结果生成行程]
    E --> F[生成后检查与补充]
    F --> G[项目保存]
```

↓

多轮修改 / 恢复 / 导出 / 查看记忆

其中工具规划由大模型完成，工具计划校验器负责兜底，工具执行器负责按顺序执行工具并解析工具之间的依赖参数。

3.2 已接入工具

当前工具体系包括：

工具	作用
map_tool	查询目的地地图信息、城市、adcode、经纬度
weather_tool	根据出行日期判断实况天气、短期预报或超范围提示
budget_tool	根据天数、人数、预算档位估算旅行预算
poi_tool	查询真实景点、餐饮、商圈等 POI 信息
route_tool	计算 POI 之间距离，辅助路线合理性判断
image_tool	根据最终行程地点匹配对应图片
hotel_tool	根据核心景点分布推荐住宿区域和酒店候选
transport_tool	根据出发地和目的地生成城际交通建议
opening_hours_tool	检查景点开放时间和闭馆风险
event_tool	查询出行期间城市活动与特殊事件

3.3 状态管理与记忆能力

项目已经具备项目级记忆系统，主要包括：

memory.constraints	用户持续约束
memory.avoid_places	明确避免地点
memory.preferences	用户偏好记忆
version_history	行程版本历史

典型能力包括：

用户说“不去外滩”，后续修改时持续生效
用户提出偏好后，后续生成和修改时注入 Prompt
用户觉得上一版更好时，可以恢复上一版
用户可以查看当前项目记忆和版本历史

3.4 保存与导出能力

项目保存逻辑已经从简单覆盖升级为快照机制：

首次保存 / 首次导出：生成 v1 JSON
未修改继续导出：仍覆盖当前 v1，不生成新 JSON
修改后保存 / 导出：生成 v2 JSON
v2 后继续导出：仍覆盖 v2
再次修改后保存 / 导出：生成 v3 JSON

报告导出支持：

Markdown
HTML
PDF

HTML / PDF 报告中可以展示基础信息、天气、预算、每日行程、地点图片、酒店建议、城际交通建议、开放时间风险和城市活动提醒。

3.5 Web 化能力

项目已经开始进入 Web 化阶段，完成了：

FastAPI 后端接口化
首页 / 新建行程页设计
历史项目页设计
行程详情页设计
简单 HTML + CSS + JavaScript 页面雏形

核心 API 包括：

POST /api/trips/generate	生成新行程
GET /api/trips	查看历史项目
GET /api/trips/detail	查看项目详情
POST /api/trips/edit	修改行程
POST /api/trips/restore	恢复上一版
GET /api/trips/memory	查看当前记忆
POST /api/trips/export	导出报告

四、当前项目结构复盘

当前推荐项目结构如下：

```
mini_trip_assistant/  
|  
├── main.py  
├── app.py  
└── menu.py
```

```

|—— trip_service.py
|
|—— planner.py
|—— prompts.py
|—— llm_client.py
|—— itinerary_editor.py
|
|—— tool_planner.py
|—— tool_executor.py
|—— tool_plan_validator.py
|
|—— memory_manager.py
|—— storage.py
|—— exporter.py
|—— templates.py
|—— date_utils.py
|
|—— api/
|   |—— main_api.py
|   |—— schemas.py
|   |—— trip_routes.py
|
|—— tools/
|   |—— tool_registry.py
|   |—— map_tool.py
|   |—— weather_tool.py
|   |—— budget_tool.py
|   |—— poi_tool.py
|   |—— route_tool.py
|   |—— image_tool.py
|   |—— hotel_tool.py
|   |—— transport_tool.py
|   |—— opening_hours_tool.py
|   |—— event_tool.py
|
|—— templates/
|   |—— index.html
|   |—— trips.html
|   |—— detail.html
|
|—— static/
|   |—— style.css
|   |—— app.js
|
|—— saved_trips/
|—— exports/
|—— .env
|—— requirements.txt

```

主要职责划分如下：

文件 / 模块	职责
main.py	命令行程序入口
app.py	CLI 主流程控制
menu.py	CLI 输入与菜单
trip_service.py	核心业务服务，负责生成、修改、保存、导出项目
planner.py	调用大模型生成最终行程
prompts.py	构造工具规划、行程生成、行程修改 Prompt
llm_client.py	封装大模型 API 调用
tool_planner.py	规划工具调用
tool_executor.py	执行工具计划
tool_plan_validator.py	校验和补全工具计划
memory_manager.py	项目记忆和版本历史管理
storage.py	项目快照保存与读取
exporter.py	Markdown / HTML / PDF 报告导出
date_utils.py	出行日期校验与计算
api/	FastAPI 接口层
tools/	各类外部工具能力
templates/	Web 页面模板
static/	前端静态资源

五、与官方项目对照方向

第十三章官方项目是我们后续复盘的重要参照对象。对照时不应简单照抄官方代码，而是要重点观察官方项目的工程组织方式。

5.1 项目结构对照

需要重点观察：

官方入口文件如何设计
 配置文件如何管理
 工具模块如何组织
 Agent 核心逻辑放在哪里
 前后端如何分离
 数据是否持久化

对照我们当前项目，需要检查：

trip_service.py 是否过重
tools 是否需要统一抽象
Prompt 是否需要拆分
API 层和业务层是否职责清晰
Web 页面是否只是临时页面还是可持续扩展

5.2 工具封装对照

重点观察官方 Tool 是否具有统一格式：

输入参数是否规范
输出结构是否统一
失败时是否有 error 信息
工具描述是否清楚
工具是否能被 Agent 自动选择

我们当前工具较多，但返回结构还不完全统一。后续可以统一为：

```
{  
  "tool_name": "...",  
  "success": true,  
  "data": {},  
  "error": null,  
  "note": ""  
}
```

这样前端展示、日志记录和错误处理都会更稳定。

5.3 Agent 流程对照

官方项目通常更关注 Agent 基础流程：

```
用户输入  
↓  
Agent 判断任务  
↓  
选择工具  
↓  
执行工具  
↓  
整合结果  
↓  
输出答案
```

我们的项目在此基础上增加了更多产品化能力：

记忆系统
版本恢复
保存快照
报告导出
开放时间检查
图片匹配
酒店选址
城际交通
城市活动提醒
Web API 与页面

对照时需要判断：这些扩展能力是否让系统更完整，还是让流程变得过重。部分工具后续可能需要变成可选能力，而不是每次都强制调用。

5.4 Prompt 设计对照

官方 Prompt 可能更简洁，我们当前 Prompt 规则较多，包括：

不能编造远期天气
不能编造票价、余票、航班、车次
不能忽略用户记忆
必须遵守 `avoid_places`
要参考 POI、路线、酒店、交通、活动

后续优化方向：

拆分通用规则和任务规则
生成 Prompt、修改 Prompt、修复 Prompt 分开维护
减少单个 Prompt 过长导致的不稳定
为不同场景准备不同模板

5.5 Web 端对照

需要观察官方前端如何处理：

页面组织
请求发送
加载状态
错误提示
历史记录
结果展示
工具结果可视化

我们当前 Web 端还比较基础，主要完成“输入—生成—展示—修改—导出”的闭环。后续需要提升页面体验和风险提示可视化。

六、当前项目优势

当前项目相比普通课程案例，优势主要体现在以下方面：

6.1 Agent 能力较完整

项目已经具备工具规划、工具调用、工具校验、多轮修改、记忆注入、报告导出等完整链路，能够较好体现 Agent 的基本工作方式。

6.2 真实旅行约束考虑较多

系统不只生成景点列表，还考虑了：

天气有效性
预算估算
路线距离
酒店选址
出发地交通
开放时间
城市活动
地点图片

这些能力让它更接近真实旅行规划产品。

6.3 状态管理能力较强

项目支持：

项目保存
快照版本
修改后生成新 JSON
恢复上一版
导出历史
用户记忆

这比一次性生成文本的 Demo 更接近真实应用。

6.4 已经具备 Web 化基础

项目已经完成 FastAPI 接口化和简单 Web 页面设计，为后续前端优化、页面美化、数据库接入打下基础。

七、当前项目不足

7.1 数据源稳定性不足

当前依赖的数据源包括高德 API、Unsplash、Ticketmaster 和大模型判断。存在以下问题：

P0I 图片可能不准确
开放时间可能缺失
活动数据对国内城市覆盖不足
酒店没有真实价格、评分和库存
交通没有实时车次、航班和票价
天气只能支持短期参考

因此当前结果更适合作为“规划参考”，不能作为绝对决策依据。

7.2 行程数据结构不够细

目前每日行程仍以 morning / afternoon / evening 文本为主。图片匹配、开放时间检查等功能需要从文本中提取地点，容易出错。

后续应升级为结构化行程：

```
{
  "day": 1,
  "date": "2026-05-01",
  "items": [
    {
      "time_period": "morning",
      "place_name": "武康路",
      "place_type": "街区",
      "start_time": "09:30",
      "end_time": "11:30",
      "reason": "适合拍照和城市漫步"
    }
  ]
}
```

7.3 缺少用户确认机制

当前 Agent 通常直接生成方案，但真实产品中，高影响决策应让用户确认，例如：

是否接受较远景点
是否愿意早起
是否接受高预算酒店
是否参加演唱会或展览
是否避开人多区域

后续应加入“候选方案 + 用户选择 + 再生成”的交互方式。

7.4 Web 端仍较粗糙

当前 Web 页面主要验证接口闭环，视觉效果和交互体验还不足：

缺少地图可视化
缺少生成进度提示
缺少风险颜色标记
缺少版本对比
缺少工具结果折叠展示
缺少移动端适配

7.5 缺少系统化测试集

当前主要靠手动测试。后续需要建立固定测试案例，用于验证：

是否遵守用户约束
是否正确恢复上一版
是否避免编造远期天气
修改后是否生成新快照
图片是否匹配具体地点
开放时间风险是否提示
活动信息查不到是否不编造

八、后续优化计划

8.1 第一阶段：结构化行程数据改造

优先级最高。

目标是把当前文本式行程升级为结构化行程，减少后续工具对文本解析的依赖。

计划内容：

设计 `trip_data.plan[].items` 结构
每个 item 包含地点、类型、时间、经纬度、推荐理由
修改 Prompt 输出格式
修改前端展示逻辑
让 `image_tool`、`opening_hours_tool`、`route_tool` 直接读取结构化地点

预期收益：

图片匹配更准
开放时间检查更准
路线计算更准
地图展示更容易
前端卡片展示更清晰

8.2 第二阶段：统一 Tool 返回格式

目标是让所有工具返回结构统一，便于日志、前端展示和错误处理。

统一格式建议：

```
{
  "tool_name": "weather_tool",
  "success": true,
  "data": {},
  "error": null,
  "note": ""
}
```

计划内容：

- 统一 success / error / note 字段
- 把核心结果放入 data
- 统一异常捕获
- 统一 execution_logs 记录格式
- 前端工具调试面板统一读取

8.3 第三阶段：生成进度与错误处理

当前生成完整行程会调用多个工具和大模型，耗时较长。后续需要改进用户体验。

计划内容：

- 前端显示生成中状态
- 后端记录每一步执行状态
- 工具失败时不中断整体流程
- 展示哪些工具成功、哪些失败
- 必要时引入异步任务或任务状态轮询

8.4 第四阶段：Web 页面美化与风险可视化

计划内容：

- 优化首页表单布局
- 优化行程详情卡片
- 增加风险颜色标记
- 增加记忆侧边栏
- 增加版本历史面板
- 增加工具结果折叠区
- 增加导出记录展示

重点是让用户直观看到：

哪些地方有风险
系统记住了什么
当前是第几个快照版本
哪些工具支撑了这个行程

8.5 第五阶段：测试用例集建设

建议建立固定测试案例：

测试场景	验证重点
上海 3 日游，不去外滩	是否遵守 avoid_places
杭州 2 日游，轻松节奏，雨天	天气适配与节奏控制
北京 4 日游，博物馆偏好，周一出发	开放时间与闭馆风险
成都 3 日游，低预算，美食优先	预算和偏好匹配
长距离出发地到目的地	第一天是否安排轻松
修改后导出多次	快照保存逻辑是否正确

8.6 第六阶段：数据库替代 JSON 文件

当前 JSON 文件适合学习阶段，但后续如果想更接近真实产品，可以引入数据库。

可拆分为：

trips 行程项目主表
trip_versions 行程版本表
tool_results 工具结果表
memories 用户项目记忆表
exports 导出记录表

这样可以支持更好的历史查询、分页、筛选和多用户管理。

8.7 第七阶段：主题化产品改造

后续可以做一个更有个人特色的版本，例如：

铁路沿线智能旅行助手
毕业旅行规划助手
低预算学生旅行助手
城市周末游助手

其中“铁路沿线智能旅行助手”与用户已有铁路相关项目背景较契合，可以作为作品集方向。

九、阶段性结论

当前智能旅行助手已经完成从“课程案例复现”到“Agent 产品雏形”的转变。它已经具备多工具调用、项目记忆、版本快照、报告导出、Web API 和简单页面展示能力，能够较完整体现智能体应用的开发思路。

但它距离真正成熟的旅行规划产品仍有差距，主要体现在数据源稳定性、行程结构化、用户确认机制、Web 交互体验和测试体系方面。

因此，后续不应继续盲目增加工具，而应重点推进：

结构化行程数据
统一工具返回格式
Web 风险可视化
生成进度与错误处理
测试用例集
数据库化存储
主题化产品改造

一句话总结：

当前项目已经不是简单 Demo，而是一个具备真实产品雏形的智能旅行规划 Agent；下一阶段的重点，是把它从“能跑、能生成”继续推进到“结构清晰、结果可靠、交互友好、可持续优化”。